

Я поменял только обвязку. Счёт вырос в шесть раз

Я хотел выбрать кодинг-агента для себя, поэтому прогнал семь своих рабочих конфигураций на четырёх небольших задачах со строгими hidden tests. Все справились. А потом я посмотрел на цену: одна и та же GPT-5.6-sol обошлась в \$3,77, \$5,28 и \$23,06 в зависимости от harness.

ЧТО Я ВЫБРАЛ

Остаюсь на Codex. opencode держу как запасной вариант. omp после этого теста для меня закончился.

На `gpt-5.6-sol medium` восемь прогонов стоили **\$3,77 в Codex**, **\$5,28 в opencode** и **\$23,06 в omp**. По сохранённым цифрам разница между omp и Codex равна **6,11×**. Из-за недосчитанного cache-write честнее говорить о диапазоне **5,38–6,11×**.

Все 56 прогонов прошли и видимые, и скрытые тесты. Это не доказывает, что агенты одинаково хороши. Просто на этих задачах качество никого не отсеяло, а цена отсеяла очень быстро.

Что я гонял

Сначала в стенде было шесть лёгких задач. На них все выбивали 100%, и толку от такого сравнения не было. Я их выкинул и оставил четыре свои задачи, где нужно написать рабочую систему, а не поправить пару строк:

t7 · TS формулы

движок формул таблицы: токенайзер + парсер приоритетов + функции + строгая таксономия ошибок. 61+45 тестов.

t8 · PY запросы

движок запросов: filter→sort→distinct→paginate→project, строгая семантика типов. 29+22.

t9 · TS стек-VM

интерпретатор ассемблера: стек, переменные, метки, переходы, лимит шагов. 30+33.

t10 · PY cron

планировщик cron: разбор 5 полей, правило OR для dom/dow, следующие N срабатываний. 19+22.

Скрытые тесты ловят именно те решения, которые выглядят услужливо, но нарушают условие: коэрсят типы, читают неинициализированную переменную как ноль или путают OR с AND. Я заранее проверил, что такие варианты падают, а эталон проходит. Для t10 дополнительно сверил результат с независимым brute-force сканером на 24 расписаниях. Зависимости для grader не нужны: TS проверяется через `node --test` и `tsc`, Python — через `unittest`.

СЕМЬ СВЯЗОК

● codex56

Codex · 5.6-sol · med

дешевле всех

● codex55

Codex · 5.5 · high

● oc56

opencode · 5.6-sol · med

● oc55

opencode · 5.5 · high

● omp55

omp · 5.5 · high

● cscopus

Claude Code · opus-4.8 · xhigh

● omp56

omp · 5.6-sol · med

дороже всех

Все инструменты были на подписках. Доллары ниже — не списания с карты, а пересчёт по API-тарифам. Для Codex и opencode я считал input/cache-read/output по \$5/\$0,5/\$30 за миллион токенов. Cache-write стоит \$6,25/M, но отдельного счётчика не осталось, поэтому дешёвые колонки немного занижены. Отсюда диапазон 5,38–6,11x вместо красивого, но слишком точного 6,11x. Claude Code здесь для контекста: у него другая модель.

1. По тестам ничья: 56 из 56

Каждая связка дважды решила все четыре задачи и прошла скрытую проверку. Самые быстрые укладывались в пару минут, самые медленные возились до 15.

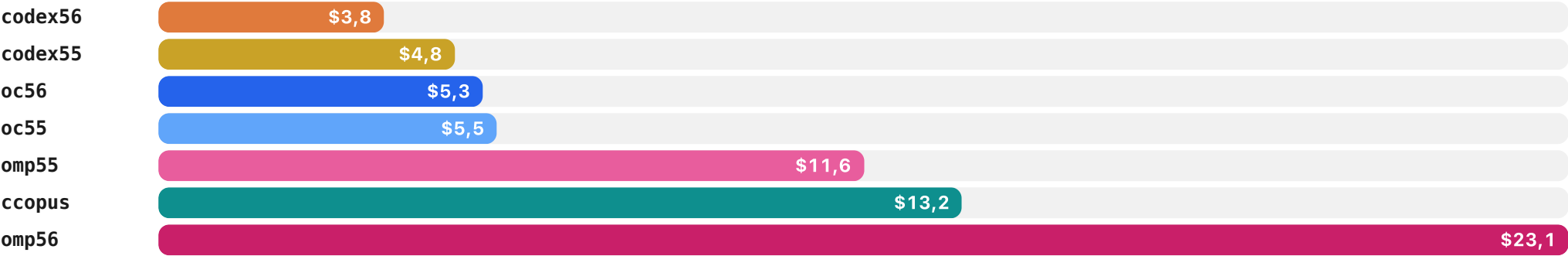
	t7 формулы	t8 запросы	t9 VM	t10 cron
● codex56	✓✓	✓✓	✓✓	✓✓
● codex55	✓✓	✓✓	✓✓	✓✓
● oc56	✓✓	✓✓	✓✓	✓✓
● oc55	✓✓	✓✓	✓✓	✓✓
● omp55	✓✓	✓✓	✓✓	✓✓
● omp56	✓✓	✓✓	✓✓	✓✓
● cscopus	✓✓	✓✓	✓✓	✓✓

На этих 56 прогонах правильность никого не разделила. Я не делаю из этого вывод, что harness одинаково хороши вообще.

2. Цена: здесь разница уже огромная

ЗАДАЧА	CODEX56	CODEX55	OC56	OC55	OMP55	CCOPUS	OMP56
t7 формулы	1,36	1,82	1,47	2,04	3,23	5,68	6,72
t8 запросы	0,80	1,09	1,24	1,16	1,72	2,68	5,81
t9 VM	0,75	0,75	1,05	1,31	3,81	2,13	5,02
t10 cron	0,86	1,11	1,51	0,99	2,82	2,75	5,51
Итого · 8 прогонов	3,77	4,77	5,28	5,50	11,58	13,24	23,06

Зелёным отмечен самый дешёвый результат на задаче. Везде это Codex.



Сумма за восемь прогонов. Codex и opencode немного недосчитаны по cache-write, но даже самая жёсткая поправка оставляет между omp56 и Codex минимум 5,38х.

3. Дорогие связки заодно оказались медленными

В первой версии я вообще убрал время: решил, что параллельный запуск сделал его бесполезным. Потом всё-таки посчитал. Порядок семи связок почти полностью совпал с порядком по цене, Pearson r = 0,986. Но у Codex часть времени восстановлена по таймстемпам, поэтому одну соседнюю пару нельзя честно расставить по местам.

ЗАДАЧА · СЕК (ПРОГОН 1 / ПРОГОН 2)	CODEX56	CODEX55	OC56	OC55	OMP55	CCOPUS	OMP56
t7 формулы	213 / 244°	254 / 292°	204 / 205	327 / 339	371 / 311	905 / 705	508 / 1153
t8 запросы	75 / 102°	104 / 111°	172 / 248	193 / 172	149 / 163	346 / 390	591 / 810
t9 VM	126 / 95	121 / 114	125 / 166	192 / 170	277 / 395	174 / 212	650 / 173
t10 cron	169 / 124	138 / 225	200 / 290	160 / 138	193 / 469	359 / 343	794 / 394
Итого · 8 прогонов, мин	19,1°	22,6°	26,8	28,2	38,8	57,2	84,5

° — время восьми ячеек Codex на t7/t8 пришлось восстановить по таймстемпам рабочих каталогов: scratchpad из /private/tmp уже исчез. Такая реконструкция занижает результат. На ячейках, где остались оба источника, разница составляла 10–65 секунд, в среднем 40. Поэтому 55% времени codex56 и 56% времени codex55 — не прямой замер. Плюс все прогоны шли параллельно (–P4/–P6), так что абсолютные секунды немного шумят.

0,986

корреляция цены и времени; шесть мест из семи выдерживают поправку

\$0,20–0,30

примерная цена минуты в этих прогонах

4,4×

Codex — 19 минут, opencode — 27, omp — 85 на одной модели

Я не утверждаю, что время само по себе определяет цену. Скорее оба показателя растут из-за длинных агентных циклов. Но секундомер оказался полезным сигналом: если агент работает в четыре раза дольше, счёт тоже, скорее всего, будет неприятным.

ЧТО БУДЕТ, ЕСЛИ ПОПРАВИТЬ ВРЕМЯ CODEX

Я добавил к каждой восстановленной ячейке сначала среднюю ошибку, потом максимальную:

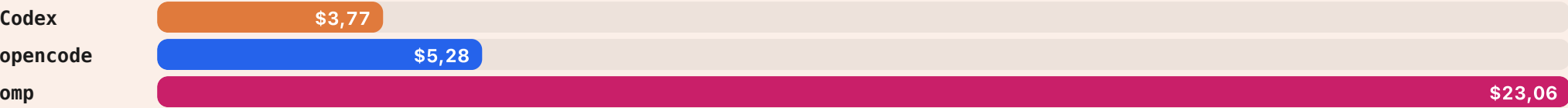
- Без поправки все семь мест совпадают с ценой.
- При +40 секундах на ячейку порядок не меняется.
- При +65 секундах codex55 и oc56 меняются местами с разницей в девять секунд.

Итого: шесть мест из семи устойчивы. Кто быстрее между codex55 и oc56, этот тест не доказал. Зато крупные разрывы никуда не деваются: omp56 работает больше чем в четыре раза дольше Codex, Claude Code с Opus — примерно втрое.

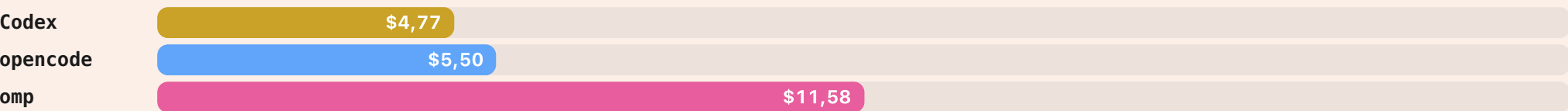
4. Одна модель, три обвязки

Ради этого сравнения я и затевал весь тест. Модель и effort одинаковые, меняется только harness. Все задачи решены, но цена расходится в разы.

GPT-5.6-SOL · MEDIUM — ТА ЖЕ МОДЕЛЬ, 8 ПРОГОНОВ



GPT-5.5 · HIGH — ТА ЖЕ МОДЕЛЬ, 8 ПРОГОНОВ



Обе шкалы нарисованы в одном масштабе. Для 5.6 medium разница между Codex и omp лежит где-то между 5,38× и 6,11×. Нижняя граница уже включает максимально невыгодную поправку cache-write для Codex.

ОМР ДОРОЖЕ НА КАЖДОЙ ЗАДАЧЕ, А НЕ ТОЛЬКО В СУММЕ

ЗАДАЧА	CODEX	OPENCODE	OMP	OMP / CODEX
t7 формулы	\$1,36	\$1,47	\$6,72	4,92×
t8 запросы	\$0,80	\$1,24	\$5,81	7,25×
t9 VM	\$0,75	\$1,05	\$5,02	6,69×
t10 cron	\$0,86	\$1,51	\$5,51	6,44×
итого	\$3,77	\$5,28	\$23,06	6,11×

Во всех строках одна модель: `gpt-5.6-sol medium`. Отношения 4,9–7,3× немного завышены из-за cache-write у Codex. По сумме после предельной поправки остаётся 5,38×.

5. Дело не только в болтливости

Я сначала подумал, что дорогие агенты просто печатают больше текста. Не сходится. Если вычесть стоимость output tokens, основная разница всё равно остаётся.

СВЯЗКА	\$ ВСЕГО	\$ OUTPUT	NON-OUTPUT RESIDUAL	ДОЛЯ RESIDUAL	OUTPUT TOKENS
codex56	3,77	1,70	2,07	55%	56 741
ос56	5,28	1,32	3,96	75%	43 962
сcopus	13,24	5,12	8,12	61%	204 874
omp56	23,06	2,97	20,09	87%	99 067

Residual — итоговая оценка минус стоимость выходных токенов. Разложить остаток на свежий input, чтение и запись кэша, служебный контекст и число ходов уже нельзя: этих счётчиков нет.

Лучше всего это видно на codex56 и ос56. opencode напечатал на 23% меньше токенов, чем Codex: 43 962 против 56 741. При этом его оценка на 40% выше. Значит, длина ответа разницу не объясняет.

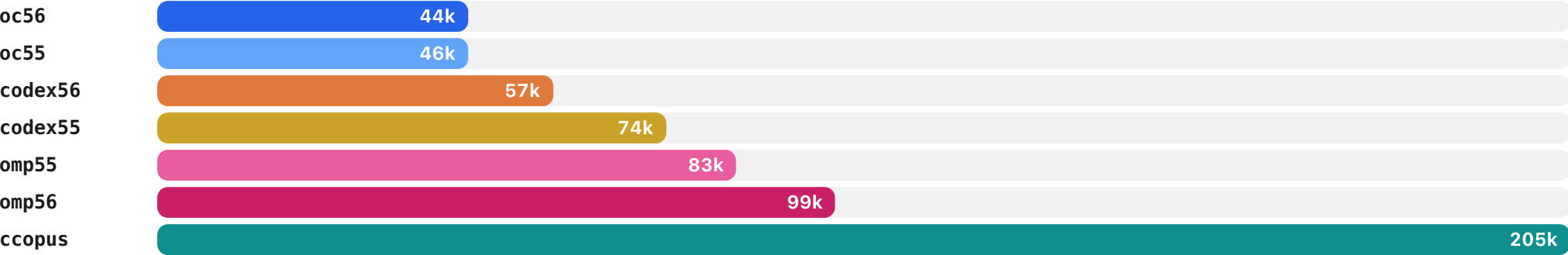
У omp56 выходные токены стоят только \$2,97 из итоговых \$23,06. Остальные \$20,09, то есть 87%, сидят в части, которую сохранённый датасет уже не умеет разложить.

ДАЖЕ ПОРЯДОК МОДЕЛЕЙ ЗАВИСИТ ОТ HARNESS

В omp профиль 5.6 medium стоит \$23,06, почти вдвое дороже 5.5 high за \$11,58. В Codex всё наоборот: 5.6 medium дешевле 5.5 high, \$3,77 против \$4,77. Здесь одновременно меняются модель и effort, поэтому причину я не изолировал. Но практический вывод простой: нельзя выбрать «дешёвый профиль» отдельно от harness.

6. Если уйду с Codex, пойду в opencode

opencode оказался хорош. По цене и скорости он рядом с Codex, а выходных токенов выдаёт меньше всех. Если я перестану пользоваться моделями GPT или Claude, то спокойно перееду сюда: подписка opencode Go за \$10 даёт доступ к основным open source моделям. После поправки cache-write разница с Codex может ещё сократиться. Оба всё равно очень далеко от omp.



Выходные токены за восемь прогонов. opencode самый короткий. Claude Code с Opus напечатал примерно в 4,5 раза больше, хотя тесты прошли все.

7. Где я ошибся и что в данных неполно

CODEX НЕ ПРЫГАЛ ПО ЦЕНЕ. ЭТО БЫЛ МОЙ КРИВОЙ ЗАПУСК

В прошлом черновике я написал, что цена codex56 скачет от \$0,6 до \$3,1. Оказалось, флаг `-s mcp_servers={}` не отключал MCP из глобального конфига. Подвисавший `exa` раздувал отдельные сессии в 2–6 раз. Я вынес Codex в отдельный чистый `CODEX_HOME` и переснял всё заново. После этого codex56 оказался самым ровным и дешёвым.

- Изоляция.** Codex работал с чистым `CODEX_HOME` без MCP, opencode — с `--pure`, omp — с `--no-lsp` через `openai-codex`. Реальных web/MCP-вызовов в прогонах не было.
- Цена.** Codex и opencode пересчитаны по общей формуле $\$5/\$0,5/\$30$. Cache-write недосчитан. omp отдаёт свою цену, у opencode в OAuth-логах она нулевая. Ошибочный fallback \$0,125 для одной ячейки я заменил на \$0,4479: одни только выходные токены там стоят \$0,1596.
- Автономность.** Все 56 прогонов прошли headless и ни разу не спросили человека. Каждый агент сам дошёл до зелёных тестов.
- Время.** Я зря убрал его из первой версии. Крупные разрывы переживают и параллельный запуск, и ошибку реконструкции. Но 55% итогового времени codex56 восстановлено, а не замерено напрямую.
- Выборка.** Четыре задачи и два прогона — это личный срез, а не доказательство на все случаи. Я доверяю разнице в разы. Мелкие дельты здесь ничего не значат. Effort не выровнен, потому что я сравнивал свои реальные конфигурации.

Что делаю по итогу

- **Codex остаётся моим рабочим инструментом.** Когда вендор делает и модель, и harness под неё, это видно. Связка Codex + GPT быстрая, экономная и просто работает.
- **opencode — запасной аэродром.** Если придётся слезть с иглы Codex + GPT, я уже знаю куда идти: opencode Go за \$10 и open source модели.
- **omp оказался переоценённой хутой.** У меня нет более аккуратной формулировки. Конфигурация 16.5.2 с gpt-5.6 medium проиграла на каждой задаче и по цене, и по времени. Это вывод ровно про протестированную конфигурацию, но для моей рутины вопрос закрыт.
- **Claude Code с Opus — дорого и долго.** Даже с xhigh effort. Для сравнения, Codex на GPT-5.5 high отрабатывал заметно быстрее. Из-за другой модели это не чистый тест harness, но выбирать эту связку для себя я не стал.

Окружение: OpenAI Codex v0.144.4 · opencode 1.17.17 · omp 16.5.2 · Claude Code v2.1.210 · Node 26.5.0 · Python 3.14.6 · TS 5.9.3. 56 прогонов (7 связок × 4 задачи × 2), метрики восстановлены из логов харнессов (ssusage-codex + пересчёт по токенам, claude-json, потоки событий opencode/omp). Это не рейтинг продуктов, а мой выбор инструмента на моём стеке. В репозитории лежат задачи t7–t10 и датасет с цифрами. Разброс 6,11× — верхняя граница; с максимально невыгодной поправкой cache-write остаётся 5,38×.